# 1. Relatório completo do Tech Challenge na AWS

## Implementação, comandos, alterações e evidências

Projeto FIAP Tech Challenge Fase 2 | Conta 637246457028 | us-east-2 (Ohio) | Consolidação em 22/10/2026, America/Sao_Paulo.

A ingestão Bronze, o replay de streaming e as camadas Silver e Gold foram implantados e executados. Este documento explica como o trabalho foi realizado, o que mudou nos notebooks recebidos, quais regras foram aplicadas e onde estão as evidências. O pacote inclui os códigos completos, parâmetros, testes e relatórios JSON.

| Componente | Resultado comprovado |
|---|---|
| Bronze batch | 7 tabelas; 3.902.954 registros somados; contagens origem/destino iguais. |
| Streaming simulado | 30 eventos únicos, 5 duplicatas e 1 inválido; 30 objetos Bronze e inválido na DLQ. |
| Silver | 7 tabelas; 3.902.954 registros; leitura de um único lote Bronze aprovado. |
| Gold | 5 datasets; 46.342 linhas somadas; conjunto publicado por manifesto SUCCEEDED. |
| FinOps | Sem upgrade ou agenda nova; reserva sugerida de US$ 30 para 30 dias com um ciclo diário. |

Atenção para a análise: uma de duas linhas da validação nacional ficou fora da tolerância informativa de 0,1 p.p. A execução concluída comprova os contratos e bloqueios implementados; não significa concordância perfeita entre todos os indicadores. A divergência permanece como assunto de investigação.

Este relatório foi produzido a partir dos arquivos preservados. Não executou nova carga, não reconfigurou a AWS e não iniciou cobrança de processamento Glue nesta entrega. Os estados operacionais são os observados nas datas indicadas.

Leitura sugerida: arquitetura e alterações nas páginas 2 a 9; histórico, monitoramento, acesso e custos nas páginas 10 a 13; comandos e validação nas páginas 14 a 18; operação, rastreabilidade e referências nas páginas 19 e 20.

# 2. Arquitetura implantada e método de trabalho

O caminho batch usa os dados públicos do INEP disponíveis no BigQuery. A Bronze preserva a origem; a Silver normaliza e traduz códigos; a Gold constrói indicadores e dados para uso analítico. Cada etapa identifica a execução de origem e mantém histórico no S3.

**BATCH OFICIAL**




**REPLAY IDENTIFICADO COMO SIMULACAO**




O streaming é um replay controlado de uma amostra real do snapshot Bronze. Ele demonstra chegadas incrementais, reentrega, deduplicação e tratamento de inválidos. Não é uma conexão ao vivo com atualizações do INEP e não foi unido aos indicadores oficiais da Gold.

| Decisão | Como foi aplicada |
|---|---|
| Acesso à AWS | Console autenticado e CloudShell; boto3/CLI com a identidade da sessão. Não foram exportadas credenciais. |
| Leitura das entradas | Notebooks e enunciado tratados como fontes de projeto; pedidos do usuário definiram o trabalho autorizado. |
| Adaptação local | Inspeção das células, contratos e dependências; scripts Glue e notebooks sem outputs herdados. |
| Implantação e execução | Código enviado ao bucket de assets; jobs manuais controlados; início explícito das execuções. |
| Conferência | Estado Glue, manifestos, contagens, esquemas, inventário S3 e metadados Parquet independentes. |

Data lake: fiap-tc-fase2-2026. Assets: aws-glue-assets-637246457028-us-east-2. Role Glue existente: AWSGlueServiceRole-FIAP-BigQuery. Silver/Gold reutilizaram essa role e leram apenas o S3.

# 3. Bronze batch e preservação da origem

O notebook camada_bronze.ipynb foi convertido em bronze_glue.py e camada_bronze_glue.ipynb. A conexão Glue existente Bigquery connection foi reutilizada; o dataset de origem é basedosdados.br_inep_avaliacao_alfabetizacao, com parentProject fiap1-512214.

| Tabela | Origem | S3 Bronze | Parquet |
|---|---|---|---|
| alunos | 3.867.999 | 3.867.999 | 4 |
| municipio | 23.995 | 23.995 | 1 |
| uf | 145 | 145 | 1 |
| meta_alfabetizacao_brasil | 3 | 3 | 1 |
| meta_alfabetizacao_uf | 81 | 81 | 1 |
| meta_alfabetizacao_municipio | 10.704 | 10.704 | 1 |
| dicionario | 27 | 27 | 1 |

Para cada tabela: leitura do BigQuery, persistência temporária do DataFrame, contagem da origem, adição de id_ingestao/data_ingestao/tabela_origem, escrita Snappy Parquet e nova leitura do S3 para contar o destino. O cache evita reler a fonte apenas para a segunda ação sobre o mesmo lote.

A escrita usa errorifexists e recusa um destino já ocupado. O caminho contém ingestion_date e run_id. Tabelas vazias são recusadas no padrão; divergência de contagem ou ausência de Parquet bloqueia o sucesso. Os campos originais não foram substituídos por descrições nesta camada.

O manifesto final contém table_metrics, is_full_batch=true e all_counts_match=true. O nome table_metrics foi respeitado na adaptação da Silver; a suposição inicial de um campo tables foi corrigida antes de sua execução. Um piloto UF não é um lote completo de sete tabelas.

Job fiap-bronze-inep-batch: 2 G.1X, timeout 30 min, retries 0, concorrência 1, Glue 5.1. Execução Glue: 146 s e 293 DPU-segundos. A duração interna do manifesto é 94,407 s; ela mede outro trecho e não substitui o tempo de execução cobrado pelo serviço.

# 4. Silver e alterações no notebook recebido

O original 02_camada_silver.ipynb foi preservado. Suas células pressupunham Databricks, dbutils, Unity Catalog e Delta. A adaptação substituiu a leitura de tabelas do catálogo por sete caminhos exatos de um manifesto Bronze aprovado e a publicação por Parquet versionado no S3.

| Antes | Alteração aplicada e finalidade |
|---|---|
| Widgets dbutils | Argumentos do job/CONFIG no notebook; caminhos, limiar e catálogo explícitos. |
| Histórico lido como tabela | Um run_id de Bronze por execução; não unir snapshots e deduplicar a união por acidente. |
| Normalização depois da deduplicação | Trim, vazio para null e normalização dos identificadores antes de escolher a linha por chave. |
| Empate de timestamp arbitrário | Última data_ingestao com desempate pelo SHA256 da linha normalizada. |
| Município/UF sem descrições | Decodificação de série e rede nas duas tabelas, atendendo as colunas exigidas pela Gold. |
| Checks após gravar | Regras antes da publicação; releitura de cada saída e manifesto SUCCEEDED ao final. |

IDs de município são validados como numéricos não zero de até sete dígitos e padronizados com zeros à esquerda. UF é normalizada em letras maiúsculas. Chaves nulas, descrições desconhecidas para códigos presentes, valores numéricos inválidos e taxas fora de [0,100] bloqueiam a continuidade.

Foram realizadas nove traduções pelo dicionário: série/rede em município, UF e alunos, mais presença, preenchimento do caderno e alfabetização em alunos. As nove coberturas medidas foram 100%. O lado direito de cada lookup precisa ser único; a cardinalidade da esquerda é preservada.

Neste snapshot, a Silver removeu zero linhas por duplicidade em todas as sete tabelas. As contagens finais iguais às da Bronze são uma medição real, não uma ausência de tratamento. O dicionário também bloqueia significados conflitantes; a política genérica de empate é determinística, não um bloqueio universal de conflitos.

# 5. Semântica dos alunos e dos indicadores

O dicionário foi conferido antes de criar as flags. Presença 1 significa Presente e 0 Ausente; preenchimento 1 significa Prova preenchida e 0 Prova não preenchida; alfabetizado 1 significa Sim e 0 Não. O código de série 2 corresponde ao 2º ano do Ensino Fundamental.

Avaliado exige presença e caderno preenchido. Para um aluno avaliado, a classificação de alfabetização precisa ser conhecida e o peso deve ser positivo e finito. Nulos de origem continuam nulos quando aplicável; desconhecido não vira automaticamente false e uma meta ausente não vira zero.

### Cálculo ponderado usado na Gold

```
universo = alunos avaliados com classificação conhecida
numerador = soma(peso_aluno dos alfabetizados nesse universo)
denominador = soma(peso_aluno de todos os avaliados elegíveis)
taxa_calculada = 100 * numerador / denominador
# Denominador zero permanece sem resultado; não fabricar taxa zero.
```

Exemplo didático: dois alunos alfabetizados com peso 2 cada e um não alfabetizado com peso 2 produzem 4/6 × 100 = 66,67%. Esse exemplo ensina a fórmula; não é o indicador nacional medido no projeto.

Município, ano, série e rede precisam permanecer no mesmo grão das comparações. Uma média simples das taxas municipais não é, automaticamente, o indicador Brasil. Metas municipais e estaduais foram comparadas somente aos recortes compatíveis; a série nacional foi explicitada como 2.

As janelas temporais conservam série e rede. Variação anual é calculada somente entre anos consecutivos. A coluna resultado menos meta é expressa em pontos percentuais; -5 p.p. significa cinco pontos abaixo da meta. Meta oficial pactuada para 2030 não foi substituída por uma hipótese de universalização.

# 6. Gold e os cinco datasets produzidos

| Dataset | Linhas | Grão e finalidade |
|---|---|---|
| indicador_municipio | 23.995 | Município × ano × série × rede; taxa oficial, recálculo e diferença. |
| resultado_vs_meta | 10.948 | Nível territorial × ano × recorte; taxa, metas e situação. |
| evolucao_temporal | 52 | UF/Brasil × ano × série/rede aplicável; evolução entre anos consecutivos. |
| validacao_cruzada_brasil | 2 | Brasil × ano × recorte elegível; taxa recalculada versus publicada. |
| features_alfabetizacao | 11.345 | Município × série × rede × ano_features; atributos t e alvo t+1. |

O original 03_camada_gold.ipynb foi convertido em gold_glue.py e em notebook Glue. dbutils, Delta, volumes e nomes de catálogo Databricks foram substituídos por parâmetros, contrato de manifesto e caminhos S3. Não há fallback silencioso para Bronze se a Silver não estiver pronta.

O script valida as sete entradas Silver, contagens, tipos e um único run_id de origem. Todas as construções e regras precedem o manifesto final; cada dataset escrito é relido e validado. Os caminhos são exclusivos da execução e os nomes Gold no Data Catalog são versionados por rodada.

| Situação da meta | Significado |
|---|---|
| atingiu | Resultado e meta existem, e resultado é maior ou igual à meta. |
| nao_atingiu | Resultado e meta existem, e resultado é menor que a meta. |
| sem_meta | Não há meta comparável disponível para a linha. |
| sem_resultado | Há meta, mas não há resultado oficial comparável disponível. |

A distinção sem_resultado foi adicionada para não chamar ausência de resultado de ausência de meta. Nenhum resultado oficial foi imputado para fazer uma junção atingir o limiar.

# 7. Coberturas das junções e divergências analíticas

| Junção construída | Casamentos / elegíveis | Cobertura |
|---|---|---|
| Município × microdados | 23.193/23.995 | 96,658% |
| Resultado municipal × meta | 10.654/10.896 | 97,779% |
| Resultado UF × meta | 49/49 | 100,000% |
| Brasil × taxa oficial | 2/2 | 100,000% |
| Features t × alvo t+1 | 11.345/11.547 | 98,251% |
| Features × meta contextual | 5.302/5.396 | 98,258% |

O limiar bloqueante continua em 95%. A cobertura é medida antes da junção e exige unicidade das chaves à direita para não contar multiplicação de linhas como correspondência. Universos elegíveis são definidos por ano/rede/série aplicável, sem filtrar previamente pelos municípios ou UFs que casaram.

Cobertura total e elegível são medidas diferentes. Por exemplo, metas não se aplicam a todos os anos e redes de uma tabela. Coberturas informativas totais permanecem no JSON; o relatório não chama 100% de um universo elegível de 100% de todas as referências possíveis.

### Diferença entre regra bloqueante e informativa

A tolerância de 0,1 p.p. entre taxa recalculada e oficial é informativa no código implantado. Em indicador_municipio, 22.468 de 23.193 linhas comparáveis ficaram nessa tolerância (96,874%). Em validacao_cruzada_brasil, apenas 1 de 2 linhas ficou nessa tolerância (50%).

Logo, o status SUCCEEDED não prova que as duas taxas nacionais coincidem. A linha nacional divergente deve ser investigada com universo, pesos, disponibilidade e definição do indicador, sem alterar resultados para forçar concordância. O detalhe do valor divergente não está no extrato local de linhas; é consultável no Parquet Gold correspondente.

# 8. Primeira falha da Gold e correção da regra

A primeira execução da Gold foi bloqueada de verdade. Ela não foi uma demonstração sintética com DEMO_FAILURE e não publicou um conjunto Gold aprovado. O relatório dessa tentativa, o código implantado e a auditoria failed.json permanecem preservados.

| Etapa | Resultado e interpretação |
|---|---|
| Tentativa inicial | Meta UF -> resultado UF: 49 de 54 referências elegíveis tinham resultado (90,741%), abaixo de 95%. |
| Referência completa | 49 de 81 referências tinham resultado. Havia 5 lacunas UF/ano nos anos observados e 27 referências em outro(s) ano(s). |
| Correção v1.0.1 | Resultado oficial publicado -> sua meta comparável. As ausências na direção inversa continuam como diagnóstico. |
| Tentativa final | 49/49 resultados UF elegíveis têm referência; cobertura 100%. Municipal: 10.654/10.896, ou 97,779%. |

O contrato de consistência passou a exigir referência para cada resultado oficial existente. A presença de uma referência de meta não autoriza criar um resultado oficial ausente. O limiar de 95% foi mantido e a elegibilidade não foi restrita somente às geografias já casadas.

Não se afirma que todas as 27 referências de outros anos sejam futuras. Também não se conclui que a cobertura final representa a completude de todo o universo territorial esperado. A alteração está no sentido e no universo da verificação, não nas fontes nem nos valores do indicador.

Tentativa FAILED: jr_737aa335fc338309fb383cb2ac7874b3d1f05f609e607e4e64e40758d6c7576e

Tentativa SUCCEEDED: jr_74dca58f19d0bd28dd27bba71b3c1a0bc420e55ec2dab74fb047e23b1901d4cd

Custo computacional estimado da tentativa bloqueada: 249 DPU-segundos × US$ 0,44 / 3.600 = US$ 0,030433. O insucesso não apaga o custo nem a evidência histórica.

# 9. Streaming simulado e garantias de processamento

Fluxo implantado: amostra Parquet alunos da Bronze aprovada -> produtor Python -> SQS Standard -> Lambda Python 3.12 -> JSON bruto e claims no S3. O envelope declara simulation=true, source=inep_bronze_replay, source_snapshot_run_id, event_id, run_id e event_time UTC.

| Medição da demonstração | Resultado |
|---|---|
| Envios para SQS | 36: 30 novos + 5 duplicatas + 1 inválido. |
| Recebimentos Lambda | 40: 30 aceitos + 5 duplicatas + 5 tentativas do inválido. |
| Persistência | 30 objetos Bronze, 30 claims; 33.759 bytes de eventos Bronze. |
| Inválido | Preservado na DLQ após cinco recebimentos malsucedidos; fila principal vazia na verificação. |
| Latência event_time -> fim do PUT | Média 0,962 s; mediana 0,117 s; p95 4,967 s; máxima 7,468 s. |

A identidade é o par (run_id,event_id). Claim e objeto Bronze usam PUT condicional, sem overwrite. Um retry recupera o claim e completa a gravação; um envelope idêntico é duplicata. O mesmo ID com conteúdo diferente é rejeitado.

SQS Standard entrega pelo menos uma vez e não garante ordenação. A idempotência protege o efeito no S3; isso não transforma o serviço em mensageria exatamente uma vez. ReportBatchItemFailures permite confirmar válidos e repetir somente os itens que falharam no lote.

Configuração: fila com visibilidade 60 s, retenção 4 dias e long polling 20 s; DLQ 14 dias e maxReceiveCount=5. Lambda com 256 MB, timeout 10 s, concorrência reservada 2, lote máximo 5 e logs por 14 dias. O produtor terminou e não há emissão recorrente configurada.

Não foi feita compactação contínua nem incorporação do replay na Silver/Gold oficiais. A amostra é proveniente de dados reais, mas os horários de chegada são da simulação e não a data do fato educacional.

# 10. Histórico no S3 e fronteira de publicação

Os caminhos abaixo são modelos de organização. Cada consumidor deve selecionar os target_uri do manifesto aprovado, e não ler todos os diretórios históricos em conjunto.

### Padrões de dados e auditoria

```
s3://fiap-tc-fase2-2026/
  bronze/inep/<tabela>/ingestion_date=<data>/run_id=<id>/
  silver/inep/<tabela>/process_date=<data>/run_id=<id>/
  gold/inep/<dataset>/ingestion_date=<data>/run_id=<id>/
  bronze/inep_streaming/entity=alunos/event_date=<data>/
    run_id=<id>/event_id=<id>.json
  auditoria/bronze/ingestion_date=<data>/run_id=<id>/
  auditoria/silver/process_date=<data>/run_id=<id>/
  auditoria/gold/ingestion_date=<data>/run_id=<id>/
  auditoria/streaming/runs/run_id=<id>/
```

Uma escrita S3 não é uma transação de múltiplas tabelas. Se uma tentativa falha, podem existir objetos ou registros parciais. A fronteira que autoriza consumo do conjunto é o manifesto final SUCCEEDED, escrito depois das validações. Não há alias latest na Gold.

O catálogo Silver usa tech_challenge_fase2_silver, com sete entradas atualizadas após validação e identificação de ownership. A Gold usa tech_challenge_fase2_gold, com nomes por execução. Atualizar várias entradas do Data Catalog também não é atômico; o manifesto continua sendo a referência do conjunto.

A linhagem é Bronze jr_cdfd2933220ec75b45c04518c6e5afe93eb98915f3b88328f8bde8d7183d3700 -> Silver jr_9c58c50a3e452ab6fff7f8560524b5235aa0d0c92036feccc6768ce2499b0288 -> Gold jr_74dca58f19d0bd28dd27bba71b3c1a0bc420e55ec2dab74fb047e23b1901d4cd.

Os três manifestos completos e seus hashes estão nos arquivos de evidência do pacote. A Bronze usa table_metrics; a Silver e a Gold publicam seu contrato de tabelas e qualidade. Os verificadores não assumem que todos os JSON têm o mesmo formato.

# 11. Monitoramento e interpretação dos alarmes

| Alarme | Condição |
|---|---|
| fiap-inep-streaming-dlq | Mensagem visível na DLQ >= 1. |
| fiap-inep-streaming-rejected | FIAP/Streaming Failed >= 1 em 60 s. |
| fiap-inep-streaming-age | Idade da mensagem mais antiga >= 120 s. |
| fiap-silver-inep-failure | FIAP/Medallion Failed >= 1, camada silver. |
| fiap-gold-inep-failure | FIAP/Medallion Failed >= 1, camada gold. |

Os scripts Silver e Gold emitem logs no formato EMF com Layer e JobName. O streaming emite Received, Accepted, Duplicate, Failed e LatencySeconds. Durações, status, contagens e caminhos também são persistidos em JSON no S3; prints isolados não foram considerados evidência suficiente.

A falha Gold de 22/10 gerou métrica Failed=1 às 14:06 UTC. O histórico registra OK -> ALARM às 14:07:56 UTC (11:07:56 em São Paulo). Isso comprova avaliação real da métrica, sem SetAlarmState artificial.

O retorno ALARM -> OK ocorreu às 14:13:56 UTC por ausência de dados tratados como não violação, conforme TreatMissingData=notBreaching. Ele precede o término da Gold final às 14:14:12 UTC. Portanto, o retorno a OK não foi usado como prova de sucesso; essa prova vem do Glue, do manifesto e do verificador.

Os cinco alarmes são acompanhados no console. Não foi configurado e-mail, SMS, webhook ou assinatura SNS. A pessoa responsável precisa consultar os alarmes e o histórico. Os logs Lambda têm retenção de 14 dias; os grupos Glue observados não tinham retenção explícita definida na consulta de custos.

A latência da demonstração foi medida até o retorno bem-sucedido do PUT. A conferência por S3 LastModified é outra medição, com resolução de segundos; ela não deve ser confundida com a latência EMF mais precisa.

# 12. Acesso de colegas e configuração de segurança

O pedido de colaboração foi analisado dentro do teto de US$ 140. O console do projeto informou que IAM Identity Center não estava disponível sem upgrade e ativação de recursos avançados. A tela de ativação indicava uma mudança irreversível. Essa ativação não foi realizada.

| Item | O que efetivamente aconteceu |
|---|---|
| FIAP-Editores | Política e revisão de escopo preparadas localmente; grupo e permission set não foram criados nem aplicados na AWS. |
| Alternativa do plano atual | Convite para r***@fiap.com.br , restrito ao projeto Try Catch Finally. |
| Evidência do convite | AWS confirmou envio em 20/10 às 16:09, UTC-3; captura preservada mostra Pendente e validade de 14 dias. |
| Aceite do colega | A captura arquivada comprova envio, não aceite. A visão geral posterior mostrou dois membros, sem uma nova captura nominal de aceite neste pacote. |
| Permissões do convite | Acesso de desenvolvedor informado pela interface; não equivale ao grupo personalizado de permissões proposto. |

A role Lambda criada pelo CloudFormation permite polling apenas da fila de origem, escrita/leitura nos prefixos streaming autorizados e escrita no grupo de logs do consumidor. O produtor usa a identidade autenticada da sessão; não há access key embutida.

A role Glue já existente foi reutilizada. Esta continuação não estreitou suas permissões e não declara que ela atende ao menor privilégio ideal. Alterar código de Glue ou Lambda permite operar com as permissões da role de execução; isso precisa ser considerado ao conceder edição.

Nenhum segredo Google ou AWS foi incluído nos scripts. env.example é referência de parâmetros; Glue não carrega um arquivo .env automaticamente. A credencial BigQuery permanece na conexão/segredo existente, sem seu valor neste relatório.

# 13. Custos medidos e projeção para trinta dias

| Execução | Tempo Glue | DPU-segundos | Estimativa computação |
|---|---|---|---|
| Bronze | 146 s | 293 | US$ 0,035811 |
| Silver | 258 s | 517 | US$ 0,063189 |
| Gold final | 136 s | 272 | US$ 0,033244 |
| Gold bloqueada | 124 s | 249 | US$ 0,030433 |

Tarifa regional conferida para Ohio: US$ 0,44 por DPU-hora. Fórmula: DPUSeconds / 3.600 × 0,44. As três rodadas de 22/10 (Silver, Gold bloqueada e Gold final) somam 1.038 DPU-segundos e US$ 0,126867. Um ciclo completo bem-sucedido Bronze+Silver+Gold soma 1.082 DPU-segundos e US$ 0,132244.

| Cenário em 30 dias | Ciclos | Glue | S3 armazenamento[1] |
|---|---|---|---|
| 4 rodadas no mes | 4 | US$ 0,53 | US$ 0,018 |
| 1 rodada por dia | 30 | US$ 3,97 | US$ 0,098 |
| 4 rodadas por dia | 120 | US$ 15,87 | US$ 0,376 |

[1] S3 conservador: bytes atuais dos dois buckets (266.974.790) mais 144.067.311 bytes Parquet por novo ciclo, todos mantidos pelo mês inteiro, a US$ 0,023/GB-mês. Não inclui pedidos ou transferência. Não é uma fatura; crescimento uniforme custaria menos em armazenamento.

Para um ciclo diário, a reserva sugerida é US$ 30 para o mês, com margem para CloudWatch, Lambda/SQS demonstrativos, segredo, pedidos S3 e testes. Não é uma medição desses custos futuros. A consulta de 22/10 mostrava US$ 137,35 em créditos, US$ 2,65 acumulados e 177 dias de plano gratuito; faturamento pode atrasar.

O job antigo fiap-ingestao-bigquery-uf tem 10 G.1X e timeout de 480 min: oito horas custariam US$ 35,20 de computação. Os jobs novos têm 2 G.1X. Se os três atingissem seus timeouts (30+15+15 min), seriam US$ 0,88 por ciclo, não os US$ 0,132 medidos. Reserva e alarmes não bloqueiam gastos automaticamente.

Nenhum agendamento foi criado. Sessões de prévia observadas estavam TIMEOUT/STOPPED; EC2 e NAT Gateway não foram encontrados em Ohio na consulta. A projeção não inventaria outras regiões, novas consultas Google Cloud, novos serviços ou treinamento pesado.

# 14. Comandos de consulta e preparação do ambiente

Os comandos desta seção são receitas compatíveis com os scripts finais e com CloudShell Bash. Os relatórios comprovam as operações e resultados; não foi preservada uma transcrição integral de cada tecla da sessão. Assim, este caderno não se apresenta como um histórico literal completo do terminal.

Extraia o ZIP, entre na pasta raiz e use uma sessão AWS autenticada na conta do projeto. Os comandos não contêm credenciais. As linhas longas podem quebrar visualmente no PDF; copie-as de Comandos_Comentados_AWS.txt. A documentação desta entrega não executou esses comandos na AWS.

### Somente leitura: identidade e inspeção dos jobs

```
aws sts get-caller-identity
aws glue get-jobs --region us-east-2 \
  --query 'Jobs[].{Job:Name,Workers:NumberOfWorkers,Tipo:WorkerType,Timeout:Timeout}'
aws glue get-triggers --region us-east-2
aws glue list-sessions --region us-east-2
```

### Somente leitura: uso atual de S3 e catálogo

```
aws s3 ls s3://fiap-tc-fase2-2026/ --recursive --summarize
aws glue get-tables --database-name tech_challenge_fase2_silver \
  --region us-east-2 --query 'TableList[].Name'
aws glue get-tables --database-name tech_challenge_fase2_gold \
  --region us-east-2 --query 'TableList[].Name'
aws cloudwatch describe-alarms --alarm-name-prefix fiap- \
  --region us-east-2
```

### Preparação local: ambiente isolado para scripts SDK

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-verificacao.txt
# Nao inicia Spark nem job Glue.
# Registre as versoes instaladas para sua nova reproducao:
python -m pip freeze > dependencias_reproducao.txt
```

Na verificação CloudShell desta execução foram utilizados boto3 1.43.115 e PyArrow 26.0.0. O consumidor Lambda usa Python 3.12 e biblioteca padrão; jobs Spark usam o runtime Glue 5.1. A identidade do CloudShell tem as permissões da sessão do console; não criar access keys para seguir esta receita.

# 15. Comandos da Bronze e da conferência histórica

### Criação do job Bronze: comando de implantação, não de consulta

```
python deploy_bronze.py \
  --script-file bronze_glue.py \
  --source-job fiap-ingestao-bigquery-uf \
  --job-name fiap-bronze-inep-batch \
  --region us-east-2
```

Esse helper faz upload do script com hash e cria o job usando a role/conexão existentes. Ele recusa job já existente. Como o job já foi implantado na conta atual, não é necessário recriá-lo. Implantar e iniciar processamento são ações diferentes.

### Nova execução: gera custo e outro run_id

```
aws glue start-job-run --job-name fiap-bronze-inep-batch \
  --region us-east-2
# Guarde JobRunId retornado. Nao reutilize um ID antigo.
```

### Somente leitura: conferir a Bronze histórica já concluída

```
python verify_bronze.py \
  --run-id jr_cdfd2933220ec75b45c04518c6e5afe93eb98915f3b88328f8bde8d7183d3700 \
  --date 2026-10-20 --region us-east-2 \
  --output conferencia_bronze_historica.json \
  --csv contagens_bronze_historica.csv
```

### Somente leitura: copiar o manifesto histórico

```
aws s3 cp \
  's3://fiap-tc-fase2-2026/auditoria/bronze/ingestion_date=2026-10-
  20/run_id=jr_cdfd2933220ec75b45c04518c6e5afe93eb98915f3b88328f8bde8d7183d3700/manifest.json' \
  manifesto_bronze_consulta.json --region us-east-2
```

O verificador Bronze compara manifesto, estado Glue e inventário S3; a releitura Spark de contagem foi feita dentro do job. A validação adicional de footers Parquet foi usada nos verificadores Silver/Gold. Não atribuir a um verificador uma leitura que ele não implementa.

# 16. Comandos de implantação da Silver e da Gold

### Implantar Silver: cria o job, sem iniciar execução

```
python deploy_continuacao.py --layer silver \
  --directory silver --defaults-json silver/job_defaults.json \
  --output deployment_silver_novo.json
```

### Implantar ou atualizar Gold: altera configuração, sem executar

```
python deploy_continuacao.py --layer gold \
  --directory gold --defaults-json gold/job_defaults.json \
  --output deployment_gold_novo.json --update-existing
```

O helper confere a conta, faz uploads imutáveis com SHA256, verifica o conteúdo enviado e relê a configuração salva. A atualização exige tags do projeto e ausência de execução ativa. A opção --update-existing foi usada para a correção Gold v1.0.1; para atualizar uma Silver existente, a mesma opção precisa ser explícita.

### Início explícito de uma nova Silver

```
aws glue start-job-run --job-name fiap-silver-inep-batch \
  --region us-east-2
# Aguarde SUCCEEDED e o manifesto completo antes da Gold.
```

### Início explícito de uma nova Gold com parâmetros atuais

```
aws glue start-job-run --job-name fiap-gold-inep-batch \
  --region us-east-2
```

Atenção: os parâmetros atuais da Gold apontam para o manifesto Silver validado em 22/10. Rodar uma nova Silver não troca automaticamente esse ponteiro. Para encadear um novo lote, edite --SILVER_MANIFEST_URI em gold/job_defaults.json com o manifesto aprovado recém-gerado e atualize o job antes de iniciar a Gold.

De forma análoga, a Silver aponta para a Bronze validada em 20/10. Escolher outro snapshot exige atualizar --BRONZE_MANIFEST_URI. O nome do parâmetro é parte do contrato; não usar diretório raiz de histórico como se fosse uma entrada única.

# 17. Comandos de validação da Silver e da Gold

### Somente leitura: Silver real, incluindo footers Parquet

```
python verify_continuacao.py --layer silver \
  --run-id jr_9c58c50a3e452ab6fff7f8560524b5235aa0d0c92036feccc6768ce2499b0288 \
  --output conferencia_silver_historica.json --verify-parquet
```

### Somente leitura: Gold final real, incluindo footers Parquet

```
python verify_continuacao.py --layer gold \
  --run-id jr_74dca58f19d0bd28dd27bba71b3c1a0bc420e55ec2dab74fb047e23b1901d4cd \
  --output conferencia_gold_historica.json --verify-parquet
```

### Somente leitura: tentativa Gold bloqueada

```
python verify_continuacao.py --layer gold \
  --run-id jr_737aa335fc338309fb383cb2ac7874b3d1f05f609e607e4e64e40758d6c7576e \
  --output conferencia_gold_falha.json
# O retorno de falha e esperado. Nao renomear esse estado para SUCCEEDED.
```

O verificador independente consulta o Glue, procura o manifesto e a auditoria, lista objetos e confere esquema e contagens. Com --verify-parquet, baixa um objeto por vez e soma num_rows dos footers com PyArrow; não cria sessão Spark. Ele lê dados da conta e pode gerar requisições S3, mas não escreve na AWS.

Nas duas execuções aprovadas, errors=[] e pending=[], published=true, verification_status=SUCCEEDED. As contagens independentes dos footers coincidem com target_count de todas as 12 saídas Silver/Gold.

Para uma nova execução, substitua o run_id pelo JobRunId novo. Os comandos acima consultam as execuções históricas preservadas; executá-los não prova que um processamento novo tenha ocorrido.

# 18. Comandos do streaming e testes locais

### Implantar infraestrutura: cria ou atualiza recursos e role

```
python streaming/build_stack.py
aws cloudformation deploy --stack-name fiap-inep-streaming \
  --template-file streaming/streaming_stack.json \
  --capabilities CAPABILITY_NAMED_IAM --region us-east-2
```

### Demonstração nova: envia eventos e um inválido para a AWS

```
python streaming/exercise_streaming.py \
  --output-dir streaming-evidence-nova \
  --events 30 --duplicates 5 --interval-seconds 1
# Use pasta nova. O programa preserva evidencias anteriores.
# Aguarde as cinco tentativas do invalido e sua chegada a DLQ.
```

### Somente leitura operacional: verificar a demonstração histórica

```
python streaming/verify_streaming.py \
  --producer-report streaming/evidence/producer/producer_report.json \
  --events streaming/evidence/producer/events.jsonl \
  --queue-url 'https://sqs.us-east-2.amazonaws.com/637246457028/fiap-inep-streaming' \
  --dlq-url 'https://sqs.us-east-2.amazonaws.com/637246457028/fiap-inep-streaming-dlq' \
  --invalid-message-id 622aaa22-577c-4b80-b56b-459454707967 \
  --output conferencia_streaming_historica.json
```

Esse verificador consulta mensagens da DLQ e devolve a visibilidade; não apaga mensagens. A evidência é histórica: após a retenção de 14 dias, o inválido de teste pode expirar. Uma consulta posterior não substitui o relatório original.

### Testes locais sem AWS ou Spark

```
python -m unittest discover -s silver -p 'test_*.py' -v
python -m unittest discover -s gold -p 'test_*.py' -v
python -m unittest discover -s streaming -p 'test_*.py' -v
```

### Checks estáticos adicionais, também sem AWS

```
python verification/check_contracts_independent.py
```

Conferência local final em 22/10: Silver 7 testes, Gold 16 e streaming 16, todos aprovados. Mais 12 checks estáticos de contratos foram repetidos com os hashes finais, totalizando 51 verificações locais. Não são 51 execuções de integração na AWS.

# 19. Operação, pendências e preparação para a próxima fase

Depois de uma falha: verificar JobRunState, ler failed.json/failure.json e quality.json, localizar a regra bloqueante e corrigir a causa. Preserve a rodada com falha e use um run_id novo. Se o problema foi na interpretação do universo, documente a alteração como na Gold v1.0.1, sem diminuir o limiar apenas para passar.

No streaming: consultar logs, fila e DLQ; corrigir mensagem ou consumidor antes de redrive. Preserve event_id/run_id quando a intenção for reentrega do mesmo evento. Não usar purge para apagar a evidência de teste ou esconder falhas. Um mesmo identificador com payload diferente não é uma duplicata válida.

O dataset features_alfabetizacao organiza atributos em t e alvo em t+1. Features declaradas: serie, rede, taxa_t, taxa_calculada_t, media_portugues_t, alunos_avaliados_t. Alvo: alvo_taxa_alfabetizacao.

meta_ano_alvo permanece como contexto e foi excluída da lista padrão de features por falta de prova de que estava disponível no instante t. Snapshots atuais podem conter revisões históricas. A avaliação futura deve separar períodos cronologicamente, preservar identificadores e evitar informação contemporânea do alvo.

| Pendente ou limite | Consequência prática |
|---|---|
| Divergência nacional | Investigar a linha fora de 0,1 p.p.; não declarar concordância total. |
| Integração do replay | Ainda não entra na Silver/Gold oficial; manter identificação de simulação. |
| Notificação externa | Alarmes apenas no console; e-mail/SNS/webhook não implantados. |
| Controle de gastos | Reserva não é trava; revisar custo, sessões e job antigo antes de novas rodadas. |
| Entrega acadêmica | README/vídeo/diagrama finais e sua conferência completa não são comprovados por este relatório. |
| Modelo de IA | Não houve treinamento nem validação de um modelo nesta etapa. |

Ao estudar, use primeiro o relatório, depois os scripts de cada camada e por último os JSON de auditoria. Os diffs textuais incluídos mostram as células de código dos notebooks originais comparadas ao script final; são uma comparação de adaptação, não um histórico Git.

# 20. Rastreabilidade, arquivos e referências

| Arquivo no pacote | O que comprova ou permite reproduzir |
|---|---|
| bronze_glue.py e manifesto_bronze.json | Script e notebook adaptados, parâmetros, relatório real e contagens das sete tabelas. |
| silver/ e gold/ | Originais preservados, scripts finais, notebooks Glue, defaults e testes de contrato. |
| streaming/ | Consumidor, produtor, CloudFormation, verificadores e evidência completa da demonstração. |
| continuacao/evidence/ | Relatórios reais, manifestos, hashes de implantação, tentativa bloqueada e histórico dos alarmes. |
| evidence/verification_final/ | Saídas dos testes locais atuais e 12 checks com os hashes finais. |
| alteracoes/ e SHA256SUMS.txt | Diffs das células originais versus scripts; integridade de todos os arquivos do pacote. |
| acesso_fiap/ | Escopo IAM proposto e capturas do bloqueio/convite; proposta não aplicada. |

### Hashes de referência

Bronze: 41303fa81a007db33730bc0991f2f4480a6d38e6a489cfeaffe5854a796b1d24

Silver: 020832c14c9ac4dc466120fe9fb9c15e29abed9bf603b828b2caf22356757030

Gold v1.0.1: 3bc498f022924739bfd3a892a00a000604088a8b9c1b0439d07f1b1dcabca642

Lambda: 68bc8b17c9064d8eb967ae71228799b6245d223037f2b85e706a9d0a6210c9ac

O extrato antigo dos 12 checks registrava um hash Gold anterior. Ele foi mantido como histórico. A repetição atual em evidence/verification_final usa o hash final Gold acima; os scripts implantados Silver/Gold também foram comparados aos hashes dos relatórios de deployment.

Enunciado: POSTECH - IAST - Tech Challenge - Fase 2 (1).pdf, 14 páginas. Referências de estudo: pp. 2-4 (indicadores, metas e dicionário), 5-7 (pipeline e qualidade), 8-9 (métricas e FinOps), 9-11 (evidências e reprodução), 12-13 (armadilhas e validação cruzada). Os originais enviados pelo usuário são fontes do projeto, não uma autorização independente para ações na conta.

Fontes oficiais das decisões e tarifas, consultadas no trabalho: aws.amazon.com/glue/pricing/; aws.amazon.com/s3/pricing/; aws.amazon.com/cloudwatch/pricing/; aws.amazon.com/secrets-manager/pricing/; docs.aws.amazon.com/glue/latest/dg/migrating-version-51.html; docs.aws.amazon.com/lambda/latest/dg/with-sqs.html. As premissas e preços da projeção estão também no JSON de custos.